

# INFRASTRUCTURE AS CODE WITH TERRAFORM OR CLOUDFORMATION

**Services:** VPC, EC2, RDS, S3, IAM (via Terraform or CFN)

**Goal:** Build a complete VPC + 3-tier architecture using IaC tools.

**Skills:** Reusability, version control, modularization.

**Definition:** Infrastructure as Code (IaC) with Terraform is a project that automates the deployment and management of AWS infrastructure using code. It provisions AWS services such as VPC, EC2, RDS, S3, and IAM without manual setup. The project follows a three-tier architecture with web, application, and database layers. Terraform provides consistent, scalable, and repeatable deployments while reducing manual effort and configuration errors.

**Scope of the Project:** The scope of this project is to design and deploy a three-tier architecture on AWS using Terraform. It automates the provisioning of resources such as VPC, EC2, RDS, S3, and IAM through Infrastructure as Code (IaC). The project demonstrates efficient, consistent, and secure cloud infrastructure management while supporting scalability, easy maintenance, and version-controlled deployments.

## Methodology:

### Terraform-Based Infrastructure Automation (Recommended Method)

- Uses Terraform to provision all AWS resources automatically.
- Infrastructure is defined as code and stored in version control.
- Enables consistent and repeatable deployments.
- Reduces manual errors and deployment time.
- Supports modularization and scalability.
- Suitable for production-grade cloud environments.

### Infrastructure as Code (IaC) with Terraform:

The project follows the Infrastructure as Code (IaC) approach, where AWS resources are defined and managed through Terraform configuration files. This method automates infrastructure deployment, improves consistency, and reduces manual configuration effort.

### AWS CloudFormation (Alternative Method)

- Uses AWS native Infrastructure as Code service.
- Creates AWS resources through YAML or JSON templates.
- Integrates directly with AWS services.
- Supports stack-based deployments and updates.
- Suitable for AWS-only environments.
- Provides infrastructure lifecycle management.

## Services Used in this Project:

### 1. Amazon VPC

- **Definition:** Amazon VPC (Virtual Private Cloud) allows you to create a logically isolated virtual network within AWS where you can launch and manage AWS resources securely.
- **Purpose:** Provides secure networking, IP addressing, routing, and network isolation for cloud resources.
- **Role:**
  - Hosts all AWS resources.
  - Contains public and private subnets.
  - Provides network security and traffic control.
  - Connects resources through route tables and Internet Gateway.

### 2. Amazon EC2

- **Definition:** Amazon EC2 (Elastic Compute Cloud) provides scalable virtual servers in the cloud.
- **Purpose:** Runs applications, websites, APIs, and backend services.
- **Role:**
  - Hosts the Web Tier Server.
  - Hosts the Application Tier Server.
  - Processes user requests and application logic.
  - Connects with the RDS database for data retrieval and storage.

### 3. Amazon RDS

- **Definition:** Amazon RDS (Relational Database Service) is a managed database service that simplifies database administration.
- **Purpose:** Stores and manages relational data securely and efficiently.
- **Role in this Project**
  - Acts as the Database Tier.
  - Stores application data and user information.
  - Provides automated backups and maintenance.
  - Ensures high availability and data durability.

### • Amazon S3

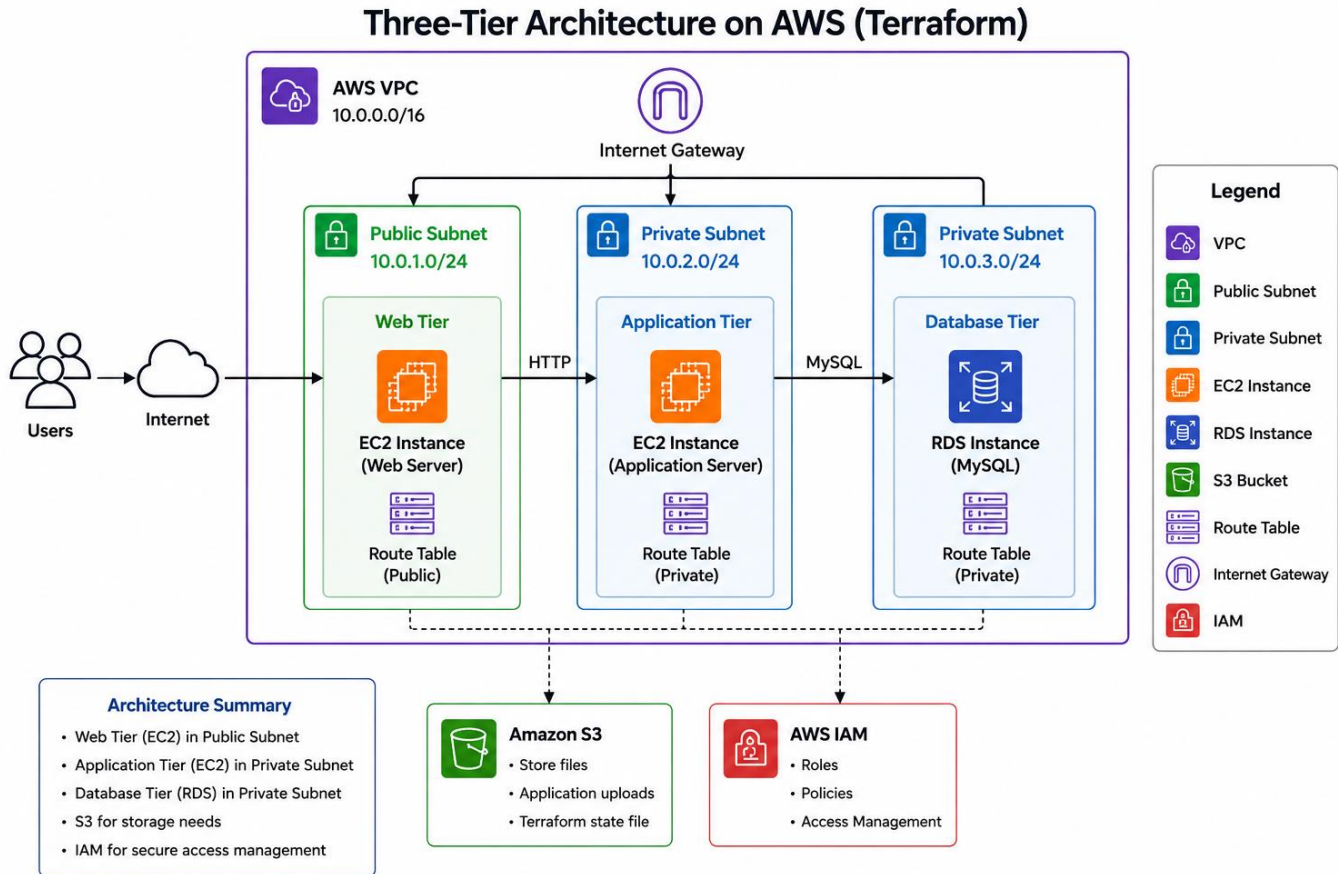
- **Definition:** Amazon S3 (Simple Storage Service) is a highly scalable object storage service.
- **Purpose:** Stores files, backups, logs, images, and application assets.
- **Role**
  - Stores application uploads and static content.
  - Stores Terraform state files (optional backend).
  - Provides durable and secure storage.
  - Supports backup and disaster recovery requirements.

### 5. AWS IAM

- **Definition:** AWS IAM (Identity and Access Management) manages authentication and authorization for AWS resources.
- **Purpose:** Controls who can access AWS resources and what actions they can perform.

- **Role:**
  - Provides secure access management.
  - Creates roles and policies for EC2 instances.
  - Grants Terraform permissions to create AWS resources.
  - Implements least-privilege security practices.

### Architecture Diagram :



## Implementation and Execution of Cloud Infrastructure Automation using Terraform and AWS

### Step 1: Launch an EC2 Server

- Instance Type: t3. micro
- Volume: 20 GB
- Region: Mumbai (ap-south-1)
- Key Pair: aws-keypair
- AMI: Amazon Linux 2023 kernel

EC2

Dashboard

AWS Global View

Events

Instances

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager

Images

AMIs

AMI Catalog

Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

Network & Security

Security Groups

Instances > i-0958f404d45088689

Instance summary for i-0958f404d45088689 (terraform-server)

Updated less than a minute ago

Instance ID

i-0958f404d45088689

IPv6 address

–

Hostname type

IP name: ip-172-31-35-124.ap-south-1.compute.internal

Answer private resource DNS name

IPv4 (A)

Auto-assigned IP address

15.206.166.197 [Public IP]

IAM role

–

IMDSv2

Required

Operator

–

Public IPv4 address

15.206.166.197 | open address

Instance state

Running

Private IP DNS name (IPv4 only)

ip-172-31-35-124.ap-south-1.compute.internal

Instance type

t3.micro

VPC ID

vpc-0604cab257558139c

Subnet ID

subnet-0474e24d37329b5ab

Instance ARN

arn:aws:ec2:ap-south-1:479897913078:instance/i-0958f404d45088689

Private IPv4 addresses

172.31.35.124

Public DNS

ec2-15-206-166-197.ap-south-1.compute.amazonaws.com | open address

Elastic IP addresses

–

AWS Compute Optimizer finding

Opt-in to AWS Compute Optimizer for recommendations. | Learn more

Auto Scaling Group name

–

Managed

false

Details

Status and alarms

Monitoring

Security

Networking

Storage

Tags

## Step 2: Install and Check the Terraform

- Connect using SSH.
- Verify successful login.
- Download Terraform package.
- Install Terraform.
- Verify installation using:

```
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

Last login: Tue Jun  2 17:28:37 2026 from 13.233.177.3
[ec2-user@ip-172-31-35-124 ~]$ sudo yum update -y
Last metadata expiration check: 0:00:13 ago on Tue Jun  2 17:29:18 2026.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-35-124 ~]$ sudo yum install -y yum-utils

sudo yum-config-manager --add-repo \
https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo

sudo yum install terraform -y
Last metadata expiration check: 0:00:30 ago on Tue Jun  2 17:29:18 2026.
Package dnf-utils-4.3.0-13.amzn2023.0.6.noarch is already installed.
Dependencies resolved.
Nothing to do.
Complete!
Adding repo from: https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
Hashicorp Stable x86_64 27 MB/s | 2.1 MB 00:00
Last metadata expiration check: 0:00:01 ago on Tue Jun  2 17:29:49 2026.
Dependencies resolved.
```

| Package                  | Architecture | Version               | Repository  | Size  |
|--------------------------|--------------|-----------------------|-------------|-------|
| Installing:              |              |                       |             |       |
| terraform                | x86_64       | 1.15.5-1              | hashicorp   | 33 M  |
| Installing dependencies: |              |                       |             |       |
| git                      | x86_64       | 2.50.1-1.amzn2023.0.1 | amazonlinux | 53 k  |
| git-core                 | x86_64       | 2.50.1-1.amzn2023.0.1 | amazonlinux | 4.9 M |
| git-core-doc             | noarch       | 2.50.1-1.amzn2023.0.1 | amazonlinux | 2.8 M |

i-0958f404d45088689 (terraform-server)

PublicIPs: 15.206.166.197 PrivateIPs: 172.31.35.124



## terraform -version

```
File      : https://rpm.releases.hashicorp.com/gpg
Key imported successfully
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing      : 1/1
  Installing     : git-core-2.50.1-1.amzn2023.0.1.x86_64 1/9
  Installing     : git-core-doc-2.50.1-1.amzn2023.0.1.noarch 2/9
  Installing     : perl-lib-0.65-477.amzn2023.0.8.x86_64 3/9
  Installing     : perl-TermReadKey-2.38-9.amzn2023.0.3.x86_64 4/9
  Installing     : perl-File-Find-1.37-477.amzn2023.0.8.noarch 5/9
  Installing     : perl-Error-1:0.17030-2.amzn2023.0.1.noarch 6/9
  Installing     : perl-Git-2.50.1-1.amzn2023.0.1.noarch 7/9
  Installing     : git-2.50.1-1.amzn2023.0.1.x86_64 8/9
  Installing     : terraform-1.15.5-1.x86_64 9/9
Running scriptlet: terraform-1.15.5-1.x86_64 9/9
  Verifying      : git-2.50.1-1.amzn2023.0.1.x86_64 1/9
  Verifying      : git-core-2.50.1-1.amzn2023.0.1.x86_64 2/9
  Verifying      : git-core-doc-2.50.1-1.amzn2023.0.1.noarch 3/9
  Verifying      : perl-Error-1:0.17030-2.amzn2023.0.1.noarch 4/9
  Verifying      : perl-File-Find-1.37-477.amzn2023.0.8.noarch 5/9
  Verifying      : perl-Git-2.50.1-1.amzn2023.0.1.noarch 6/9
  Verifying      : perl-TermReadKey-2.38-9.amzn2023.0.3.x86_64 7/9
  Verifying      : perl-lib-0.65-477.amzn2023.0.8.x86_64 8/9
  Verifying      : terraform-1.15.5-1.x86_64 9/9

Installed:
git-2.50.1-1.amzn2023.0.1.x86_64      git-core-2.50.1-1.amzn2023.0.1.x86_64
git-core-doc-2.50.1-1.amzn2023.0.1.noarch  perl-Error-1:0.17030-2.amzn2023.0.1.noarch
perl-File-Find-1.37-477.amzn2023.0.8.noarch  perl-Git-2.50.1-1.amzn2023.0.1.noarch
perl-TermReadKey-2.38-9.amzn2023.0.3.x86_64  perl-lib-0.65-477.amzn2023.0.8.x86_64
terraform-1.15.5-1.x86_64

Complete!
[ec2-user@ip-172-31-35-124 ~]$ terraform -version
Terraform v1.15.5
on linux amd64
[ec2-user@ip-172-31-35-124 ~]$
```

i-0958f404d45088689 (terraform-server)

PublicIPs: 15.206.166.197 PrivateIPs: 172.31.35.124

## Step 3: Configure AWS CLI

- Install AWS CLI.
- Verify installation.

### aws --version

```
Last metadata expiration check: 0:02:00 ago on Tue Jun 2 17:29:49 2026.
Package awscli-2.2.33-15-1.amzn2023.0.1.noarch is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-35-124 ~]$ aws --version
aws-cli/2.33.15 Python/3.9.25 Linux/6.1.172-216.329.amzn2023.x86_64 source/x86_64.amzn.2023
[ec2-user@ip-172-31-35-124 ~]$ aws configure
AWS Access Key ID [None]: AKIAW7PBPA33KUW12R5I
AWS Secret Access Key [None]: R6iCoxWq5zfBU+UJLGdvx2sVnhhWfxpypti2dPUC
Default region name [None]: ap-south-1
Default output format [None]: json
[ec2-user@ip-172-31-35-124 ~]$ ls
[ec2-user@ip-172-31-35-124 ~]$ mkdir terraform-3tier
[ec2-user@ip-172-31-35-124 ~]$ ls
terraform-3tier
[ec2-user@ip-172-31-35-124 ~]$ cd terraform-3tier/
[ec2-user@ip-172-31-35-124 terraform-3tier]$ ls
[ec2-user@ip-172-31-35-124 terraform-3tier]$ nano provider.tf
[ec2-user@ip-172-31-35-124 terraform-3tier]$ ls
provider.tf
[ec2-user@ip-172-31-35-124 terraform-3tier]$ nano variables.tf
[ec2-user@ip-172-31-35-124 terraform-3tier]$ ls
provider.tf variables.tf
[ec2-user@ip-172-31-35-124 terraform-3tier]$ nano terraform.tfvars
[ec2-user@ip-172-31-35-124 terraform-3tier]$ ls
provider.tf terraform.tfvars variables.tf
[ec2-user@ip-172-31-35-124 terraform-3tier]$ nano main.tf
[ec2-user@ip-172-31-35-124 terraform-3tier]$ ls
main.tf provider.tf terraform.tfvars variables.tf
[ec2-user@ip-172-31-35-124 terraform-3tier]$ nano outputs.tf
[ec2-user@ip-172-31-35-124 terraform-3tier]$ ls
main.tf outputs.tf provider.tf terraform.tfvars variables.tf
[ec2-user@ip-172-31-35-124 terraform-3tier]$ terraform init
Initializing provider plugins found in the configuration...
- Finding hashicorp/aws versions matching "> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)
```

i-0958f404d45088689 (terraform-server)

PublicIPs: 15.206.166.197 PrivateIPs: 172.31.35.124

## Step 4: Create Project Structure

- Run: aws configure
- Enter Access Key.
- Enter Secret Key.
- Select Region.
- Select Output Format.

```

[ec2-user@ip-172-31-35-124 ~]$ mkdir terraform-3tier
[ec2-user@ip-172-31-35-124 ~]$ ls
terraform-3tier
[ec2-user@ip-172-31-35-124 ~]$ cd terraform-3tier/
[ec2-user@ip-172-31-35-124 terraform-3tier]$ ls
[ec2-user@ip-172-31-35-124 terraform-3tier]$ nano provider.tf
[ec2-user@ip-172-31-35-124 terraform-3tier]$ ls
provider.tf
[ec2-user@ip-172-31-35-124 terraform-3tier]$ nano variables.tf
[ec2-user@ip-172-31-35-124 terraform-3tier]$ nano terraform.tfvars
[ec2-user@ip-172-31-35-124 terraform-3tier]$ ls
provider.tf terraform.tfvars variables.tf
[ec2-user@ip-172-31-35-124 terraform-3tier]$ nano main.tf
[ec2-user@ip-172-31-35-124 terraform-3tier]$ ls
main.tf provider.tf terraform.tfvars variables.tf
[ec2-user@ip-172-31-35-124 terraform-3tier]$ nano outputs.tf
[ec2-user@ip-172-31-35-124 terraform-3tier]$ ls
main.tf outputs.tf provider.tf terraform.tfvars variables.tf
[ec2-user@ip-172-31-35-124 terraform-3tier]$ terraform init
Initializing provider plugins found in the configuration...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

```

i-0958f404d45088689 (terraform-server)

PublicIPs: 15.206.166.197 PrivateIPs: 172.31.35.124

## Step 5: Create main.tf

Define VPC.

- Define Subnets.
- Define Route Tables.
- Define Internet Gateway.
- Define EC2 Instances.
- Define RDS Database.
- Define S3 Bucket.
- Define IAM Resources.

```

GNU nano 8.3 main.tf Modified

ingress {
  from_port = 3306
  to_port   = 3306
  protocol  = "tcp"
  security_groups = [aws_security_group.web_sg.id]
}

egress {
  from_port = 0
  to_port   = 0
  protocol  = "-1"
  cidr_blocks = ["0.0.0.0/0"]
}

#####
# RDS Instance
#####

resource "aws_db_instance" "mysql_db" {
  allocated_storage = 20
  engine            = "mysql"
  engine_version    = "8.0"
  instance_class    = "db.t3.micro"
  db_name           = "appdb"
  username          = var.db_username
  password          = var.db_password
  skip_final_snapshot = true
  publicly_accessible = false
  db_subnet_group_name = aws_db_subnet_group.db_subnet_group.name
  vpc_security_group_ids = [aws_security_group.rds_sg.id]
}

#####

```

i-0958f404d45088689 (terraform-server)

PublicIPs: 15.206.166.197 PrivateIPs: 172.31.35.124

## Step 6 : Create variables.tf

- Define input variables.
- Store configurable values

```
GNU nano 8.3 variables.tf Modified
variable "region" {
  description = "AWS Region"
}

variable "ami_id" {
  description = "Amazon Linux AMI"
}

variable "instance_type" {
  description = "EC2 Instance Type"
}

variable "key_name" {
  description = "EC2 Key Pair"
}

variable "instance_name" {
  description = "Web Server Name"
}

variable "db_username" {
  description = "RDS Username"
}

variable "db_password" {
  description = "RDS Password"
}
```

⌘G Help ⌘C Write Out ⌘F Where Is ⌘K Cut ⌘T Execute ⌘L Location  
⌘X Exit ⌘R Read File ⌘N Replace ⌘V Paste ⌘J Justify ⌘\_ Go To Line

i-0958f404d45088689 (terraform-server)

PublicIPs: 15.206.166.197 PrivateIPs: 172.31.35.124



## Step 7: Create terraform. Tfvars

- Store variable values.
- Customize deployment settings.

```
GNU nano 8.3 terraform.tfvars
region = "ap-south-1"
ami_id = "ami-0695bcc683dad6b9"
instance_type = "t3.micro"
key_name = "aws-keypair"
instance_name = "Terraform-Web-Server"

db_username = "admin"
db_password = "Admin12345"
```

⌘G Help ⌘C Write Out ⌘F Where Is ⌘K Cut ⌘T Execute ⌘L Location  
⌘X Exit ⌘R Read File ⌘N Replace ⌘V Paste ⌘J Justify ⌘\_ Go To Line

i-0958f404d45088689 (terraform-server)

PublicIPs: 15.206.166.197 PrivateIPs: 172.31.35.124



## Step 8: provider.tf

- Configure AWS provider.
- Set AWS region.

.

```
GNU nano 8.3 provider.tf Modified
terraform {
  required_version = ">= 1.0"

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.region
}
```

Help Write Out Where Is Cut Execute Location  
Exit Read File Replace Paste Justify Go To Line

i-0958f404d45088689 (terraform-server)

PublicIPs: 15.206.166.197 PrivateIPs: 172.31.35.124

## Step 9: Create outputs.tf

- Define output values.
- Display resource IDs and endpoints.

```
GNU nano 8.3 outputs.tf Modified
output "vpc_id" {
  value = aws_vpc.main.id
}

output "public_subnet_id" {
  value = aws_subnet.public_subnet.id
}

output "ec2_public_ip" {
  value = aws_instance.web_server.public_ip
}

output "s3_bucket_name" {
  value = aws_s3_bucket.project_bucket.bucket
}

output "rds_endpoint" {
  value = aws_db_instance.mysql_db.endpoint
}
```

Help Write Out Where Is Cut Execute Location  
Exit Read File Replace Paste Justify Go To Line

i-0958f404d45088689 (terraform-server)

PublicIPs: 15.206.166.197 PrivateIPs: 172.31.35.124

## Step 10: Initialize Terraform

- Run:
- terraform init
- Downloads required provide



```
[ec2-user@ip-172-31-35-124 terraform-3tier]$ terraform init
Initializing provider plugins found in the configuration...
- Finding hashicorp/aws versions matching "> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
[ec2-user@ip-172-31-35-124 terraform-3tier]$ terraform fmt
terraform.tfvars
[ec2-user@ip-172-31-35-124 terraform-3tier]$ terraform validate
Success! The configuration is valid.

[ec2-user@ip-172-31-35-124 terraform-3tier]$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource
actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_db_instance.mysql_db will be created
+ resource "aws_db_instance" "mysql_db" {
  + address                = (known after apply)
  + allocated_storage      = 20

```

i-0958f404d45088689 (terraform-server)

PublicIPs: 15.206.166.197 PrivateIPs: 172.31.35.124



## Step 11: Format Terraform Configuration

- Run:
- terraform fmt
- Formats Terraform files.

```
[ec2-user@ip-172-31-35-124 terraform-3tier]$ terraform fmt
terraform.tfvars
[ec2-user@ip-172-31-35-124 terraform-3tier]$ terraform validate
Success! The configuration is valid.

[ec2-user@ip-172-31-35-124 terraform-3tier]$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource
actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_db_instance.mysql_db will be created
+ resource "aws_db_instance" "mysql_db" {
  + address                = (known after apply)
  + allocated_storage      = 20
  + apply_immediately      = false
  + arn                    = (known after apply)
  + auto_minor_version_upgrade = true
  + availability_zone      = (known after apply)
  + backup_retention_period = (known after apply)
  + backup_target           = (known after apply)
  + backup_window          = (known after apply)
  + ca_cert_identifier      = (known after apply)
  + character_set_name      = (known after apply)
  + copy_tags_to_snapshot  = false
  + database_insights_mode = (known after apply)
  + db_name                 = "appdb"
  + db_subnet_group_name   = "terraform-db-subnet-group"
  + dedicated_log_volume   = false
  + delete_automated_backups = true
  + domain_fqdn            = (known after apply)
  + endpoint               = (known after apply)
  + engine                 = "mysql"
  + engine_lifecycle_support = (known after apply)
  + engine_version         = "8.0"
  + engine_version_actual   = (known after apply)
  + hosted_zone_id         = (known after apply)
  + id                     = (known after apply)

```

i-0958f404d45088689 (terraform-server)

PublicIPs: 15.206.166.197 PrivateIPs: 172.31.35.124



## Step 12: Validate Terraform Configuration

- Run:
- terraform validate
- Checks syntax and configuration errors.

```
aws [Q Search] [Alt+S]
[ec2-user@ip-172-31-35-124 terraform-3tier]$ terraform validate
Success! The configuration is valid.

[ec2-user@ip-172-31-35-124 terraform-3tier]$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource
actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_db_instance.mysql_db will be created
+ resource "aws_db_instance" "mysql_db" {
+   address                        = (known after apply)
+   allocated_storage              = 20
+   apply_immediately              = false
+   arn                            = (known after apply)
+   auto_minor_version_upgrade    = true
+   availability_zone              = (known after apply)
+   backup_retention_period        = (known after apply)
+   backup_target                  = (known after apply)
+   backup_window                  = (known after apply)
+   ca_cert_identifier             = (known after apply)
+   character_set_name             = (known after apply)
+   copy_tags_to_snapshot         = false
+   database_insights_mode         = (known after apply)
+   db_name                       = "appdb"
+   db_subnet_group_name          = "terraform-db-subnet-group"
+   dedicated_log_volume           = false
+   delete_automated_backups      = true
+   domain_fqdn                   = (known after apply)
+   endpoint                      = (known after apply)
+   engine                         = "mysql"
+   engine_lifecycle_support       = (known after apply)
+   engine_version                 = "8.0"
+   engine_version_actual          = (known after apply)
+   hosted_zone_id                = (known after apply)
+   id                            = (known after apply)
+   identifier                    = (known after apply)
+   identifier_prefix              = (known after apply)
}

i-0958f404d45088689 (terraform-server)
PublicIPs: 15.206.166.197 PrivateIPs: 172.31.35.124
```

## Step 13: Check Infrastructure Plan

```
[ec2-user@ip-172-31-35-124 terraform-3tier]$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource
actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_db_instance.mysql_db will be created
+ resource "aws_db_instance" "mysql_db" {
+   address                        = (known after apply)
+   allocated_storage              = 20
+   apply_immediately              = false
+   arn                            = (known after apply)
+   auto_minor_version_upgrade    = true
+   availability_zone              = (known after apply)
+   backup_retention_period        = (known after apply)
+   backup_target                  = (known after apply)
+   backup_window                  = (known after apply)
+   ca_cert_identifier             = (known after apply)
+   character_set_name             = (known after apply)
+   copy_tags_to_snapshot         = false
+   database_insights_mode         = (known after apply)
+   db_name                       = "appdb"
+   db_subnet_group_name          = "terraform-db-subnet-group"
+   dedicated_log_volume           = false
+   delete_automated_backups      = true
+   domain_fqdn                   = (known after apply)
+   endpoint                      = (known after apply)
+   engine                         = "mysql"
+   engine_lifecycle_support       = (known after apply)
+   engine_version                 = "8.0"
+   engine_version_actual          = (known after apply)
+   hosted_zone_id                = (known after apply)
+   id                            = (known after apply)
+   identifier                    = (known after apply)
+   identifier_prefix              = (known after apply)
+   instance_class                 = "db.t3.micro"
+   iops                           = (known after apply)
+   kms_key_id                    = (known after apply)
}

i-0958f404d45088689 (terraform-server)
PublicIPs: 15.206.166.197 PrivateIPs: 172.31.35.124
```

```
# aws_vpc.main will be created
+ resource "aws_vpc" "main" {
+   arn                            = (known after apply)
+   cidr_block                     = "10.0.0.0/16"
+   default_network_acl_id         = (known after apply)
+   default_route_table_id         = (known after apply)
+   default_security_group_id      = (known after apply)
+   dhcp_options_id               = (known after apply)
+   enable_dns_hostnames           = (known after apply)
+   enable_dns_support             = true
+   enable_network_address_usage_metrics = (known after apply)
+   id                            = (known after apply)
+   instance_tenancy               = "default"
+   ipv6_association_id            = (known after apply)
+   ipv6_cidr_block                = (known after apply)
+   ipv6_cidr_block_network_border_group = (known after apply)
+   main_route_table_id           = (known after apply)
+   owner_id                      = (known after apply)
+   tags                           = {
+     "Name" = "Terraform-VPC"
+   }
+   tags_all                      = {
+     "Name" = "Terraform-VPC"
+   }
}

Plan: 13 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ ec2_public_ip    = (known after apply)
+ public_subnet_id = (known after apply)
+ rds_endpoint     = (known after apply)
+ s3_bucket_name   = "madhu-terraform-project-bucket-2026"
+ vpc_id           = (known after apply)

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take
exactly these actions if you run "terraform apply" now.
[ec2-user@ip-172-31-35-124 terraform-3tier]$

i-0958f404d45088689 (terraform-server)
PublicIPs: 15.206.166.197 PrivateIPs: 172.31.35.124
```

## Step 14 : Deploy Infrastructure

```
[ec2-user@ip-172-31-35-124 terraform-3tier]$ terraform apply
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

- + create

Terraform will perform the following actions:

```
# aws_db_instance.mysql_db will be created
+ resource "aws_db_instance" "mysql_db" {
  + address                               = (known after apply)
  + allocated_storage                     = 20
  + apply_immediately                     = false
  + arn                                   = (known after apply)
  + auto_minor_version_upgrade            = true
  + availability_zone                     = (known after apply)
  + backup_retention_period                = (known after apply)
  + backup_target                         = (known after apply)
  + backup_window                         = (known after apply)
  + ca_cert_identifier                    = (known after apply)
  + character_set_name                     = (known after apply)
  + copy_tags_to_snapshot                 = false
  + database_insights_mode                = (known after apply)
  + db_name                               = "appdb"
  + db_subnet_group_name                  = "terraform-db-subnet-group"
  + dedicated_log_volume                  = false
  + delete_automated_backups              = true
  + domain_fqdn                           = (known after apply)
  + endpoint                              = (known after apply)
  + engine                                = "mysql"
  + engine_lifecycle_support              = (known after apply)
  + engine_version                        = "8.0"
  + engine_version_actual                  = (known after apply)
  + hosted_zone_id                        = (known after apply)
  + id                                     = (known after apply)
  + identifier                            = (known after apply)
  + identifier_prefix                     = (known after apply)
  + instance_class                        = "db.t3.micro"
  + iops                                  = (known after apply)
  + kms_key_id                            = (known after apply)
```

i-0958f404d45088689 (terraform-server)

PublicIPs: 15.206.166.197 PrivateIPs: 172.31.35.124

```
aws  [Q Search] [Alt+S] Asia Pacific (Mumbai) Madhu M (479897913078) Madhu M

# aws_vpc.main will be created
+ resource "aws_vpc" "main" {
  + arn                                   = (known after apply)
  + cidr_block                           = "10.0.0.0/16"
  + default_network_acl_id               = (known after apply)
  + default_route_table_id               = (known after apply)
  + default_security_group_id            = (known after apply)
  + dhcp_options_id                      = (known after apply)
  + enable_dns_hostnames                  = (known after apply)
  + enable_dns_support                    = true
  + enable_network_address_usage_metrics = (known after apply)
  + id                                     = (known after apply)
  + instance_tenancy                      = "default"
  + ipv6_association_id                   = (known after apply)
  + ipv6_cidr_block                       = (known after apply)
  + ipv6_cidr_block_network_border_group = (known after apply)
  + main_route_table_id                  = (known after apply)
  + owner_id                             = (known after apply)
  + tags                                  = {
    + "Name" = "Terraform-VPC"
  }
  + tags_all                             = {
    + "Name" = "Terraform-VPC"
  }
}

Plan: 13 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ ec2_public_ip   = (known after apply)
+ public_subnet_id = (known after apply)
+ rds_endpoint     = (known after apply)
+ s3_bucket_name  = "madhu-terraform-project-bucket-2026"
+ vpc_id          = (known after apply)

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes
```

i-0958f404d45088689 (terraform-server)

PublicIPs: 15.206.166.197 PrivateIPs: 172.31.35.124

```
aws db instance.mysql db: Still creating... [00m10s elapsed]
aws db instance.mysql db: Still creating... [00m20s elapsed]
aws db instance.mysql db: Still creating... [00m30s elapsed]
aws db instance.mysql db: Still creating... [00m40s elapsed]
aws db instance.mysql db: Still creating... [00m50s elapsed]
aws db instance.mysql db: Still creating... [01m00s elapsed]
aws db instance.mysql db: Still creating... [01m10s elapsed]
aws db instance.mysql db: Still creating... [01m20s elapsed]
aws db instance.mysql db: Still creating... [01m30s elapsed]
aws db instance.mysql db: Still creating... [01m40s elapsed]
aws db instance.mysql db: Still creating... [01m50s elapsed]
aws db instance.mysql db: Still creating... [02m00s elapsed]
aws db instance.mysql db: Still creating... [02m10s elapsed]
aws db instance.mysql db: Still creating... [02m20s elapsed]
aws db instance.mysql db: Still creating... [02m30s elapsed]
aws db instance.mysql db: Still creating... [02m40s elapsed]
aws db instance.mysql db: Still creating... [02m50s elapsed]
aws db instance.mysql db: Still creating... [03m00s elapsed]
aws db instance.mysql db: Still creating... [03m10s elapsed]
aws db instance.mysql db: Still creating... [03m20s elapsed]
aws db instance.mysql db: Still creating... [03m30s elapsed]
aws db instance.mysql db: Still creating... [03m40s elapsed]
aws db instance.mysql db: Still creating... [03m50s elapsed]
aws db instance.mysql db: Still creating... [04m00s elapsed]
aws db instance.mysql db: Still creating... [04m10s elapsed]
aws db instance.mysql db: Still creating... [04m20s elapsed]
aws db instance.mysql db: Still creating... [04m30s elapsed]
aws db instance.mysql db: Still creating... [04m40s elapsed]
aws_db_instance.mysql_db: Creation complete after 4m43s [id=db-PHC34PSL3TQ0334G7C567UWGO4]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:
ec2_public_ip = "13.127.216.56"
public_subnet_id = "subnet-02c7c1a2259e02c7d"
rds_endpoint = "terraform-mysql-db.cjaocmm6kkdm.ap-south-1.rds.amazonaws.com:3306"
s3_bucket_name = "madhu-terraform-project-bucket-2026"
vpc_id = "vpc-05732dc984f932b9f"
[ec2-user@ip-172-31-35-124 terraform-3tier]$
```

i-0958f404d45088689 (terraform-server)

PublicIPs: 15.206.166.197 PrivateIPs: 172.31.35.124

## Step 15: Verify Resources in AWS Console EC2 (Elastic Compute Cloud)

The screenshot displays the AWS Management Console interface for an EC2 instance. The left sidebar shows the navigation menu with categories like EC2, Images, Elastic Block Store, and Network & Security. The main content area shows the 'Instance summary' for the instance 'i-034d1b9c76763cc08' (Terraform-Web-Server). The instance is in a 'Running' state. Key details include: Public IPv4 address (13.127.216.56), Private IP DNS name (ip-10-0-1-56.ap-south-1.compute.internal), Instance type (t3.micro), VPC ID (vpc-05732dc984f932b9f), Subnet ID (subnet-02c7c1a2259e02c7d), and Instance ARN (arn:aws:ec2:ap-south-1:479897913078:instance/i-034d1b9c76763cc08). The instance is associated with the IAM role 'aws-ec2-instance-profile' and the IMDSv2 feature is required. The instance details section is expanded, showing the instance's configuration and status.

## Step 16: VPC (Virtual Private Cloud)

**VPC dashboard** < vpc-05732dc984f932b9f / Terraform-VPC

Virtual private cloud

Subnets

Route tables

Internet gateways

Egress-only Internet gateways

DHCP option sets

Elastic IPs

Managed prefix lists

NAT gateways

Peering connections

Route servers

Security

Network ACLs

Security groups

PrivateLink and Lattice

Getting started

Endpoints

Endpoint services

Service networks

**Details** info

VPC ID: vpc-05732dc984f932b9f

DNS resolution: Enabled

Main network ACL: acl-039a9165f8cfa40a

IPv6 CIDR (Network border group): -

Encryption control ID: -

State: Available

Tenancy: default

Default VPC: No

Network Address Usage metrics: Disabled

Encryption control mode: -

Block Public Access: Off

DHCP option set: dopt-080425edd2d247315

IPv4 CIDR: 10.0.0.0/16

Route 53 Resolver DNS Firewall rule groups: -

DNS hostnames: Disabled

Main route table: rtb-063b97a0099a5e7c6

IPv6 pool: -

Owner ID: 479897913078

**Resource map** info

Subnets (3)

Subnets within this VPC

ap-south-1a

Public-Subnet

Private-App-Subnet

ap-south-1b

Route tables (2)

Route network traffic to resources

rtb-063b97a0099a5e7c6

Public-RT

Network Connections (1)

Connections to other networks

Terraform-IGW

## Step 17 : Public Subnet and Private Subnet

**VPC dashboard** < Subnets

Virtual private cloud

Subnets

Route tables

Internet gateways

Egress-only Internet gateways

DHCP option sets

Elastic IPs

Managed prefix lists

NAT gateways

Peering connections

Route servers

Security

Network ACLs

Security groups

PrivateLink and Lattice

Getting started

Endpoints

Endpoint services

Service networks

**Subnets (6)** info

Find subnets by attribute or tag

| <input type="checkbox"/> | Name               | Subnet ID                | State     | VPC                              | Block Public... | IPv4 CIDR      | IPv6 CIDR |
|--------------------------|--------------------|--------------------------|-----------|----------------------------------|-----------------|----------------|-----------|
| <input type="checkbox"/> | Private-DB-Subnet  | subnet-0be1c9b8f17adacfa | Available | vpc-05732dc984f932b9f   Terra... | Off             | 10.0.3.0/24    | -         |
| <input type="checkbox"/> | Private-App-Subnet | subnet-0dc9dc6ef249bccf6 | Available | vpc-05732dc984f932b9f   Terra... | Off             | 10.0.2.0/24    | -         |
| <input type="checkbox"/> | Public-Subnet      | subnet-02c7c1a2759e02c7d | Available | vpc-05732dc984f932b9f   Terra... | Off             | 10.0.1.0/24    | -         |
| <input type="checkbox"/> | -                  | subnet-0b60b5d0720ffd1a0 | Available | vpc-0604cab257558139c            | Off             | 172.31.0.0/20  | -         |
| <input type="checkbox"/> | -                  | subnet-0474e24d37329b5ab | Available | vpc-0604cab257558139c            | Off             | 172.31.32.0/20 | -         |
| <input type="checkbox"/> | -                  | subnet-058f7538934d7e57e | Available | vpc-0604cab257558139c            | Off             | 172.31.16.0/20 | -         |

Select a subnet

## Step 18 : Internet Gateway

AWS Management Console screenshot showing the details of an Internet Gateway (igw-085ce7b84c2c52910) in the VPC dashboard. The page title is "igw-085ce7b84c2c52910 / Terraform-IGW".

**Details:**

- Internet gateway ID: igw-085ce7b84c2c52910
- State: Attached
- VPC ID: vpc-05732dc984f932b9f | Terraform-VPC
- Owner: 479897913078

**Tags (1):**

| Key  | Value         |
|------|---------------|
| Name | Terraform-IGW |

## Step 19 : Route Table

AWS Management Console screenshot showing the details of a Route Table (rtb-03e228ded2e31bb53) in the VPC dashboard. The page title is "rtb-03e228ded2e31bb53 / Public-RT".

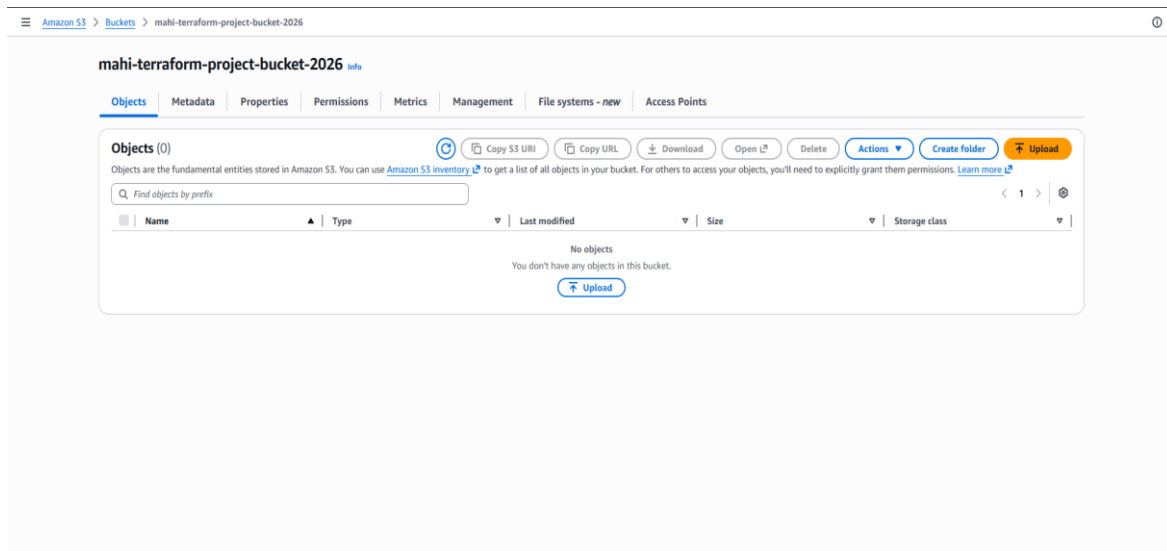
**Details:**

- Route table ID: rtb-03e228ded2e31bb53
- Main: No
- Explicit subnet associations: subnet-02c7c1a2259e02c7d | Public-Subnet
- Edge associations: -
- VPC: vpc-05732dc984f932b9f | Terraform-VPC
- Owner ID: 479897913078

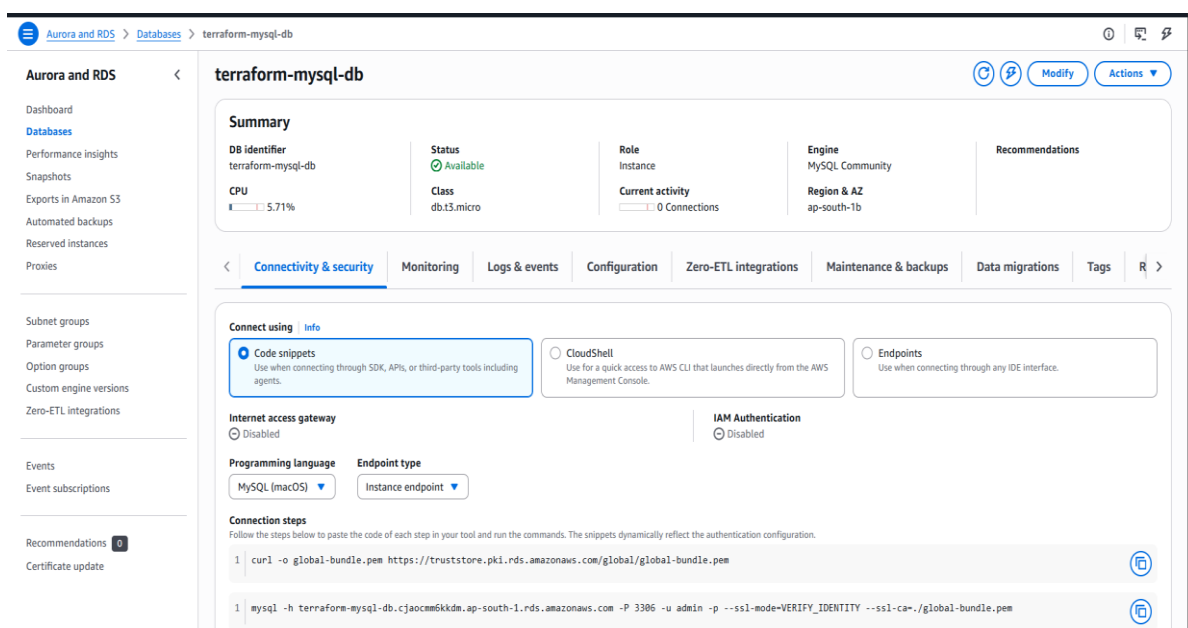
**Routes (2):**

| Destination | Target                | Status | Propagated | Route Origin       |
|-------------|-----------------------|--------|------------|--------------------|
| 0.0.0.0/0   | igw-085ce7b84c2c52910 | Active | No         | Create Route       |
| 10.0.0.0/16 | local                 | Active | No         | Create Route Table |

## Step 20 : S3 Bucket



## Step 21 : RDS (Relational Database Management)



## Troubleshooting:

### Issue 1: Terraform Initialization Failed

**Problem:** Provider plugins fail to download.

**Solution:**

- Verify internet connectivity.
- Check Terraform version compatibility.
- Run terraform init again.

### Issue 2: AWS Authentication Error

**Problem:** Terraform cannot access AWS resources.

**Solution:**

- Verify AWS Access Key and Secret Key.
- Run aws configure again.
- Ensure IAM permissions are assigned correctly.

### Issue 3: EC2 Instance Not Accessible

**Problem:** Unable to SSH into EC2 instance.

**Solution:**

- Verify key pair.
- Check Security Group port 22 access.

- Confirm instance is running.
- Verify public IP assignment.

#### **Issue 4: RDS Connection Failure**

**Problem:** Application cannot connect to database.

**Solution:**

- Verify Security Group rules.
- Check database endpoint.
- Confirm database credentials.
- Ensure subnet group configuration is correct.

#### **Conclusion:**

This project successfully demonstrated the use of Infrastructure as Code (IaC) with Terraform on AWS. By automating the deployment of resources such as VPC, EC2, RDS, S3, and IAM, a complete three-tier architecture was created in a consistent and efficient manner. The project reduced manual effort, improved infrastructure management, and enhanced scalability. It also provided hands-on experience with Terraform, AWS services, and cloud automation, serving as a strong foundation for modern DevOps practices.